

GMD2

Lecture 01: Pong

Who am I?

Jakob Trigger Knop

Cand.Scient. Comp. Science

E-mail: jknr@via.dk

Office: C03.01 (Horsens 🦴)



Who Are You?



My Expectations

Engaged students
who build cool stuff
and learn at the same time

Course Purpose

*"The course introduces the student to engine-level game programming using a low-level game framework. The student learns by implementing classic video games, gaining direct control over essential systems like game loop, rendering, asset pipeline and game physics. The purpose is to teach **software engineering in a game context**, including **architecture**, **design patterns**, **decoupling**, **component systems** and **data-driven design**."*

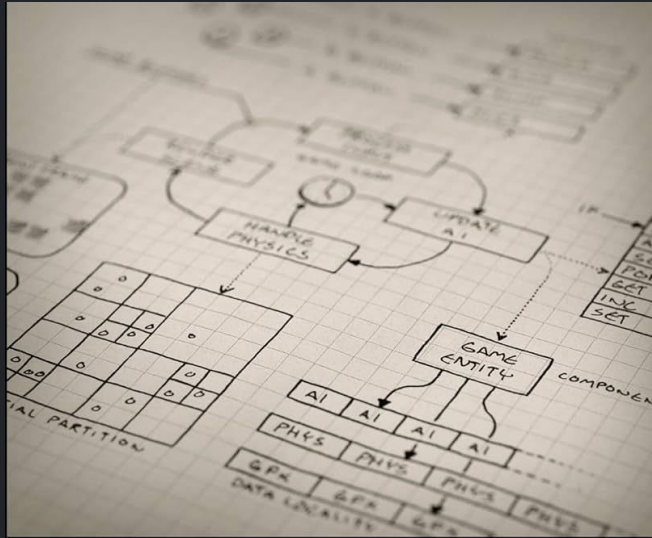
Detailed course description on itslearning

Course Topics

- Game Architecture
- Software Design Principles
- Design Patterns
 - State, Observer, Command, Component, Singleton, Service Locator, Prototype, Object Pool, etc..
- Game Loop & Rendering
- Input Handling
- Asset Pipeline
- Collision Detection
- Game Physics
- Tilemaps
- Serialization
- Data-Driven Design
- Optimization

Session Plan

1. Pong
2. Flappy Bird
3. Snake
4. Project Work
5. Super Mario Bros
6. The Legend of Zelda
7. Angry Birds
8. Project Work
9. Pokemon
10. Geometry Wars
11. Project Work
12. Wrapping Up



Game Programming Patterns

Robert Nystrom



gameprogrammingpatterns.com

Course Project

The goal of the project is to create a simple 2D game that uses best practices for organizing and architecting a game project. That is, the project must include gameplay systems and design elements solved through patterns and principles taught in the course.

Details on [itslearning](#)

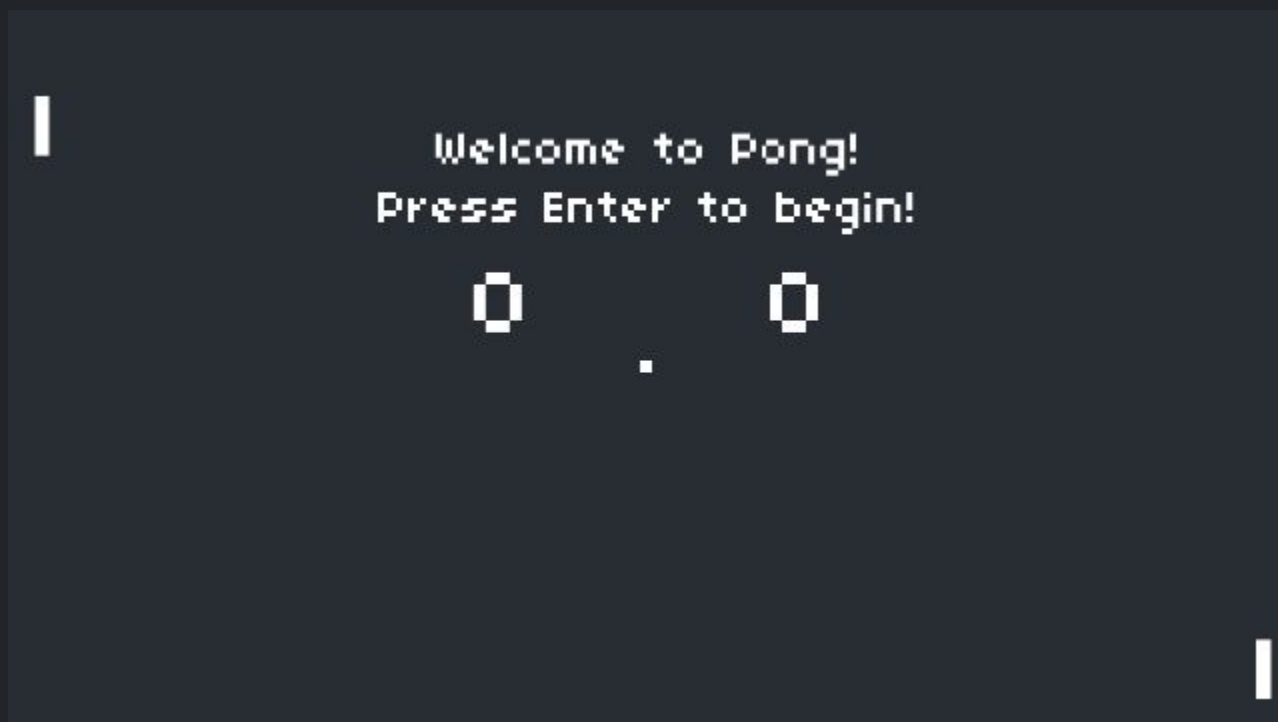
Exam

Individual oral exam, 20 minutes, without preparation.

The student will draw from a pool of known questions and is expected to reference their course project as part of the answer. This involves analyzing source code, as well as potentially writing or modifying source code during the exam. The final grade will be based on an overall assessment of the project presented and the oral examination.

Internal assessment.

Today's Goal



Today's Topics

- MonoGame & Tooling
- Game Loop
- Content Pipeline
- Drawing to the Screen
- DeltaTime & Velocity
- Game State
- Encapsulation
- Collision Detection
- Sound Effects



What is MonoGame?

At its core, MonoGame offers a set of libraries and APIs to handle common game development tasks

- Graphics Rendering
 - 2D and 3D rendering are supported through the graphics API offered by MonoGame
- Input Handling
 - Input from keyboard, mouse, gamepads, and touchscreens is supported
- Audio
 - A comprehensive audio system that can be used to create sound effects as well as play music
- Content Pipeline
 - An out-of-the-box workflow for importing and processing game assets
- Math Library
 - A math library specifically optimized for game development

Tooling

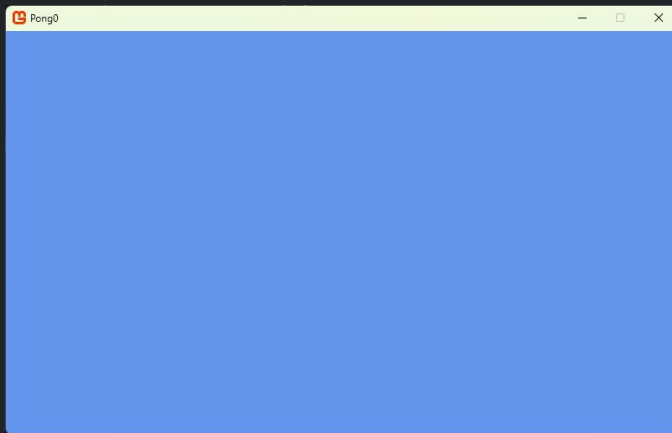
- .NET version 9+
- VScode
 - C# Dev Kit Extension
 - MonoGame for VSCode Extension
 - MonoGame Templates
 - dotnet new install MonoGame.Templates.CSharp
 - MonoGame Content Builder (Editor) Extension (Optional)

Source Code

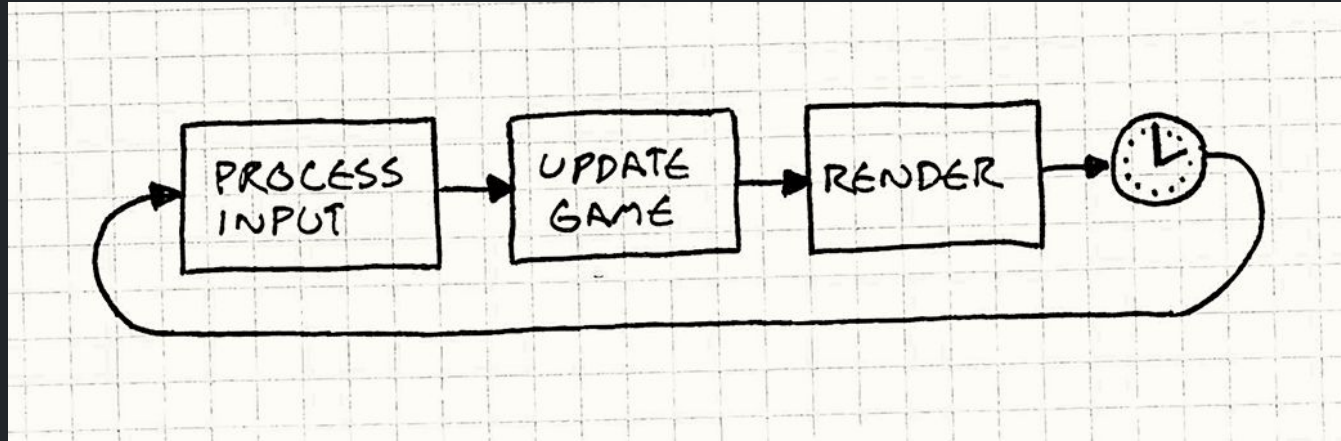
- All Source code for today's game can be found here:
 - <https://github.com/Metamate/gmd2-pong>

Pong 0: New Game

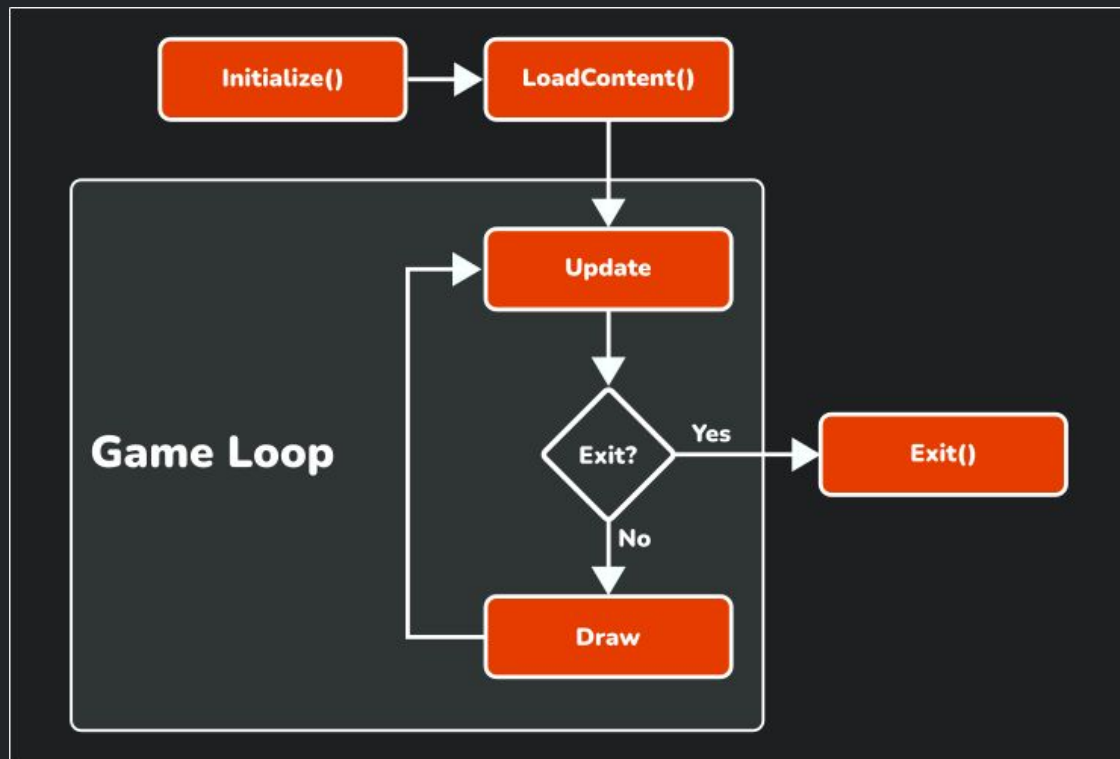
- Install the tooling from the previous slide
- Create a new MonoGame project “Pong”
 - Use the “Cross-Platform Desktop Application”-template (OpenGL)
- Run the game



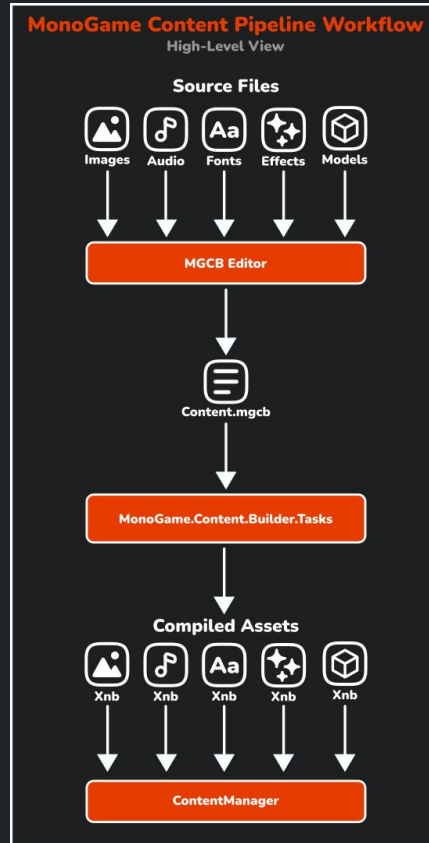
Game Loop Pattern



MonoGame Game Loop

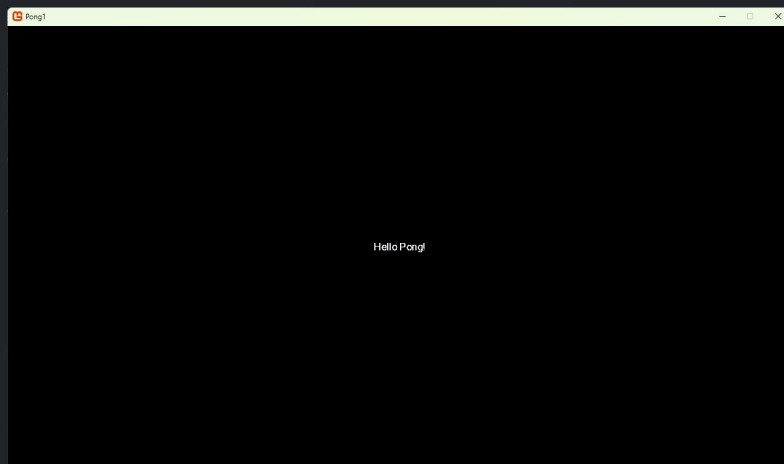


Content Pipeline

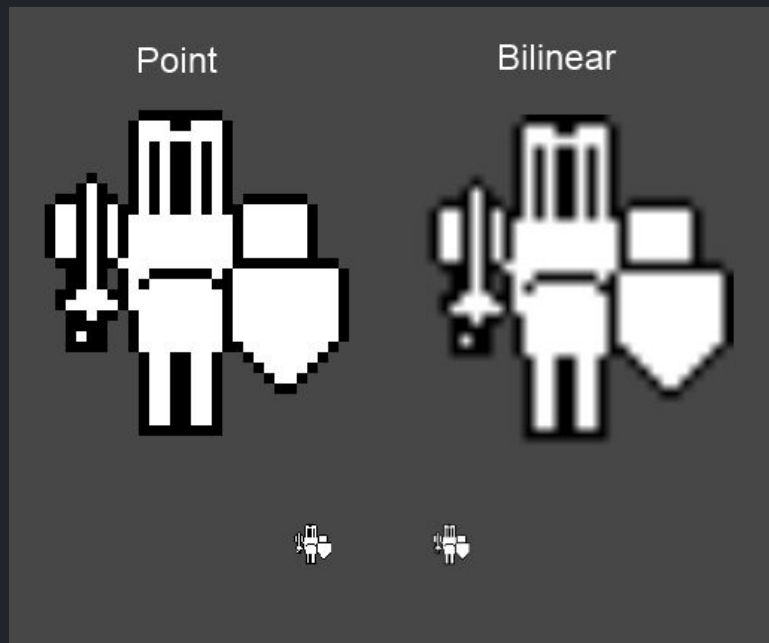


Pong 1: Hello Pong

- Update your Pong game to display “Hello Pong!” exactly in the center of the screen.
 - White text, black background
 - You will need a SpriteFont asset
 - [How to Draw Text](#)
- Set the window resolution to 1280x720
 - Use PreferredBackBuffer[Width/Height] property on GraphicsDeviceManager



Texture Filtering

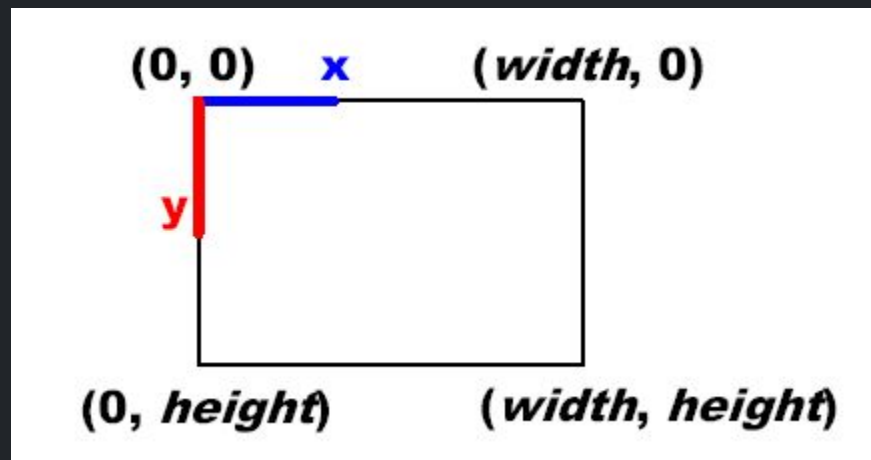


Pong 2: Virtual Resolution

- Make the game resolution independent of window resolution
- Make the game window resizable
- Update the game to use Point filtering



2D Coordinate System



Drawing to The Screen

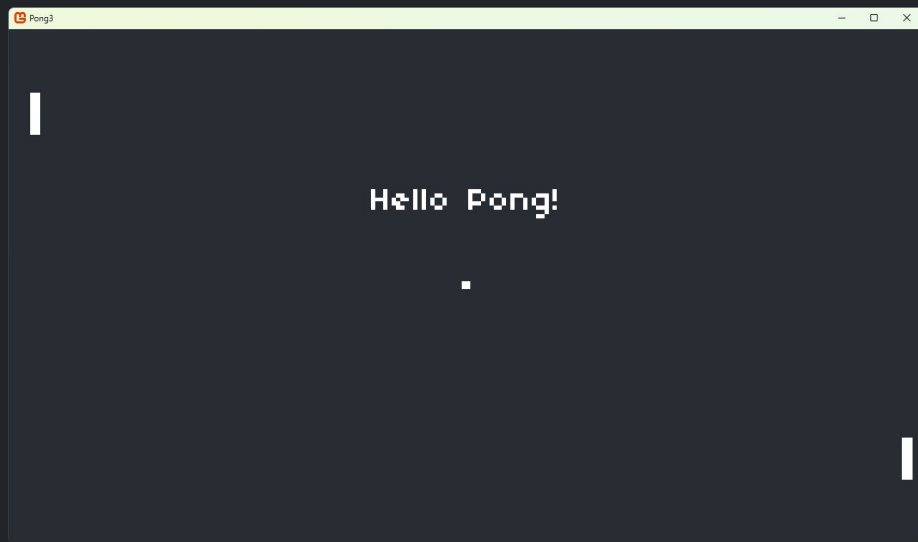
- Normally a texture is needed to draw to the screen
- A Pong game doesn't need explicit texture assets, but we do need to draw rectangles

```
private Texture2D _texture;  
...  
_texture = new(GraphicsDevice, 1, 1);  
_texture.SetData([Color.White]);
```

- Use the texture as an argument to `SpriteBatch.Draw()`
 - `SpriteBatch.Draw()` has a method overload that receives a `Rectangle`

Pong 3: Drawing Rectangles

- Draw the three rectangles present in a Pong game (Paddle, Paddle, Ball)
 - Choose appropriate starting positions for each entity
- Update the font to a custom [retro font](#)
- Update the background color to RGBA value (40, 45, 52, 255)



Basic Keyboard Input

```
KeyboardState keyboardState = Keyboard.GetState();  
  
if (keyboardState.IsKeyDown(Keys.W))  
    _player1Y -= PADDLE_SPEED * deltaTime;  
  
if (keyboardState.IsKeyDown(Keys.S))  
    _player1Y += PADDLE_SPEED * deltaTime;
```

DeltaTime & Velocity

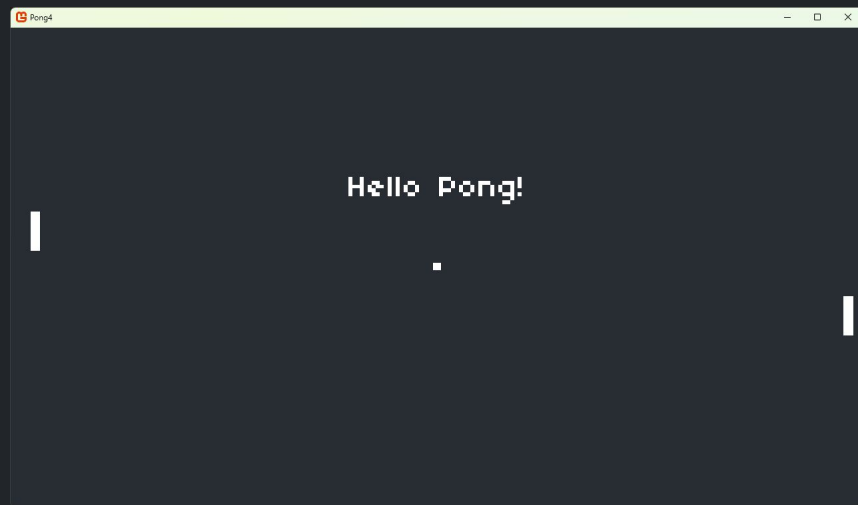
- Why do we care about deltatime?

```
(float)gameTime.ElapsedGameTime.TotalSeconds
```

- What do we need velocity for?

Pong 4: Paddle Movement

- Add interactivity by letting players move the paddles
 - W and S keys move Paddle 1 up and down
 - Up and Down arrow keys move Paddle 2 up and down
- Ensure that movement is framerate independent



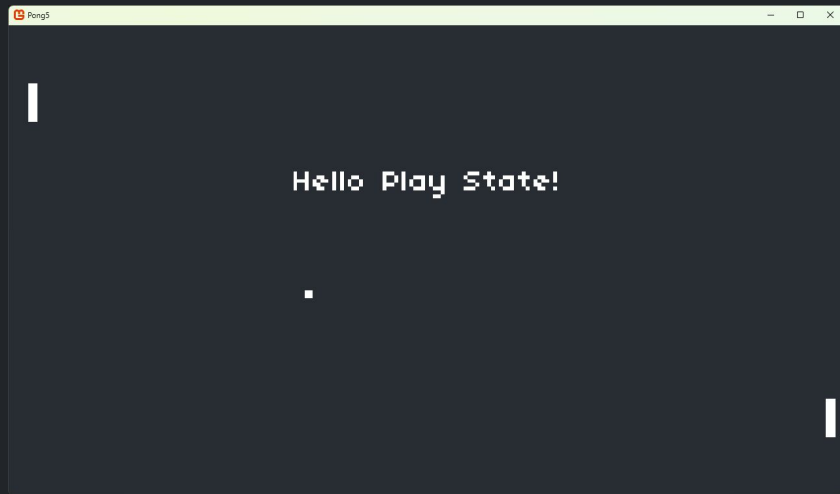
“Key Pressed”

How to emulate “GetKeyDown”?

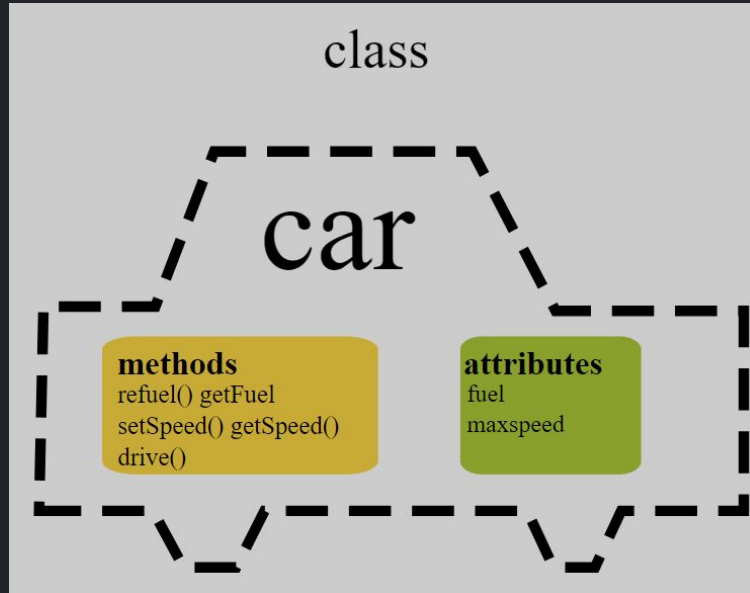
```
KeyboardState keyboardState = Keyboard.GetState();  
keyboardState.IsKeyDown(Keys.Enter) && !oldKeyboardState.IsKeyDown(Keys.Enter)  
oldKeyboardState = keyboardState;
```

Pong 5: Ball Movement

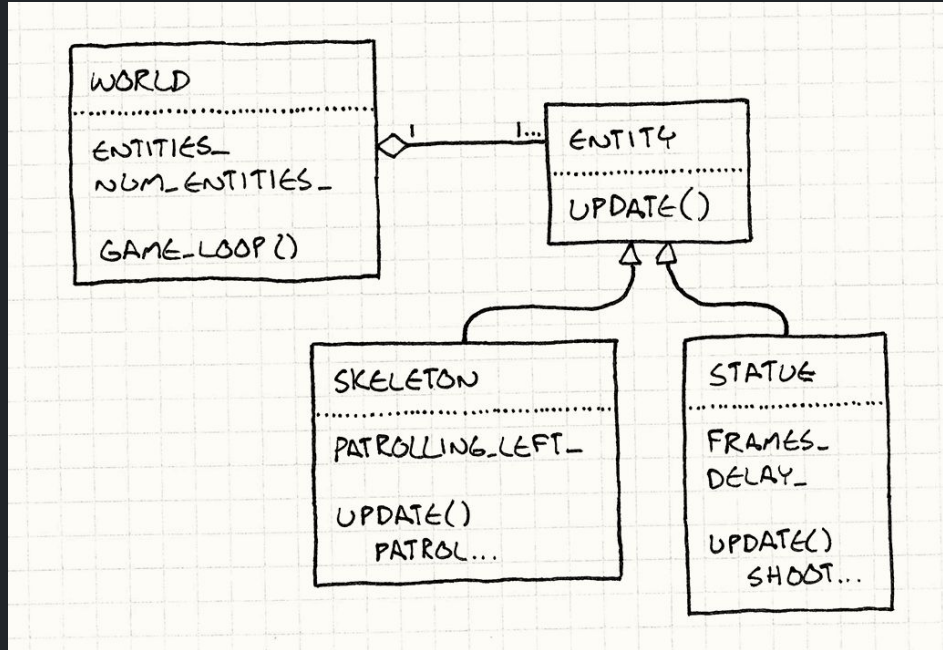
- Add motion to the ball upon the user pressing “Enter”.
 - You will need to keep track of the state of the game, e.g. using a simple string field
 - Use the Random class to ensure the ball starts with different movement vectors
- Ensure the Paddles can't move offscreen
 - `Math.Min()` and `Math.Max()` methods are helpful here



Encapsulation



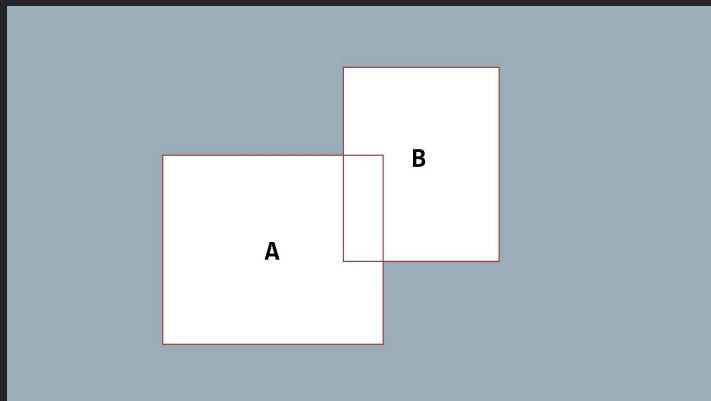
Update Method Pattern



Pong 6: Encapsulation

- Refactor the Pong code by creating two classes, Paddle and Ball, that encapsulates the data and behaviour of the two entities
 - The behaviour of the Pong game will remain identical from the previous iteration
 - You will need to take advantage of [Update Method](#) pattern

Collision Detection (AABB)



```
public static bool AABB(Rectangle boxA, Rectangle boxB)
{
    return boxA.Left < boxB.Right &&
           boxA.Right > boxB.Left &&
           boxA.Top < boxB.Bottom &&
           boxA.Bottom > boxB.Top;
}
```

Pong 7: Collision Detection

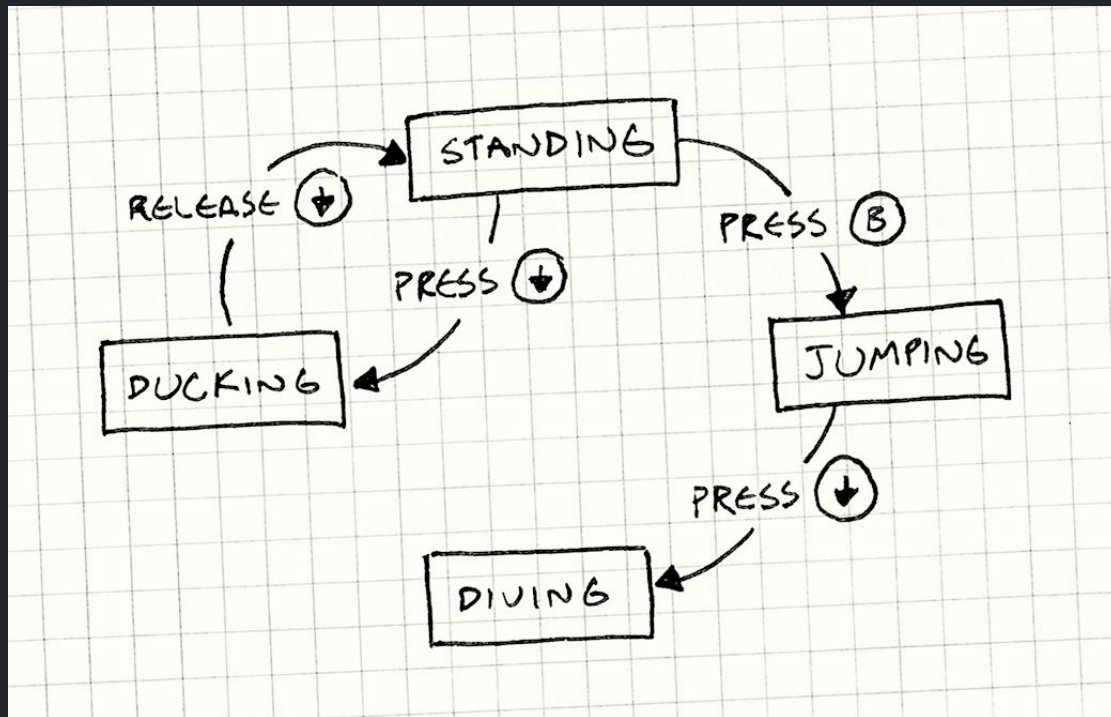
- Allow for the ball to bounce off the Paddles and window boundaries using AABB collision detection
 - Implement the Collides(Paddle paddle) method on the Ball to check for collision
 - Calculate a new random Y velocity of the Ball based on its current Y direction (up/down)
 - For every bounce on a Paddle, increase Ball X velocity by 10%

Pong 8: Scoring

- Keep track of scoring
 - The scoring UI must use a bigger font than the text

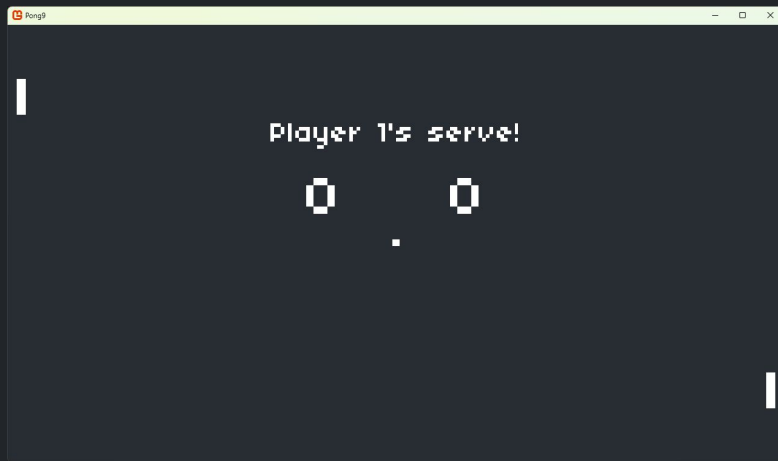


Game State



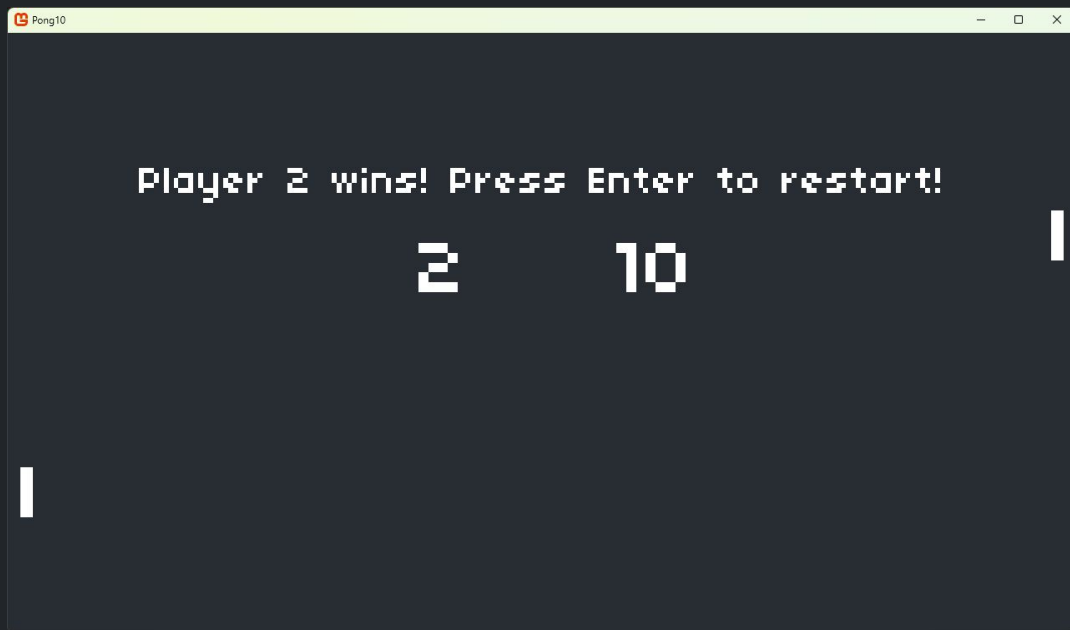
Pong 9: Serve State

- Add a new Serve state to the game
 - When a player is serving, they send the ball towards the other player
 - The player who was last scored on gets to serve (random player at game start)
 - We transition from “play” to “serve” state when scoring
 - And from “serve” state to “play” state when pressing “Enter”
 - The game begins in the “start” state and transitions to the “serve” state by pressing “Enter”



Pong 10: Victory

- Allow a player to win the game
 - Introduce a new “done” state when maximum score is reached (e.g. 10)

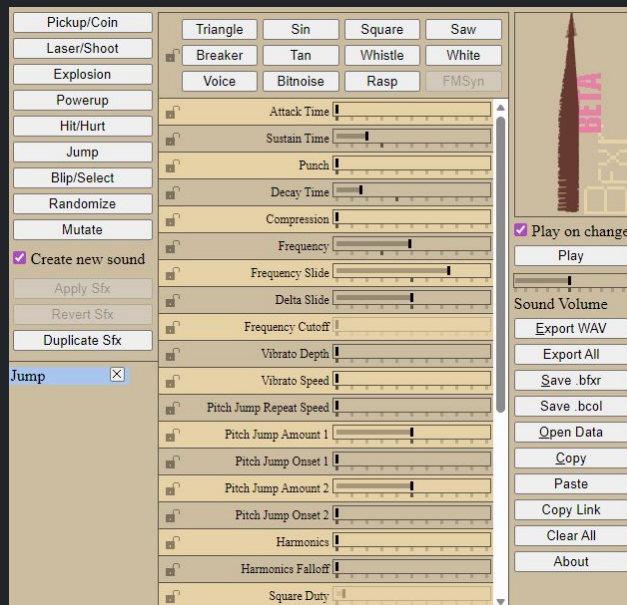


Sounds

- The [SoundEffect](#) class handles short audio clips
 - collisions, jumping, shooting, button clicks, footsteps, ambient sounds, etc.
- The key characteristics of sound effects are
 - Loaded entirely into memory for quick access
 - Can play multiple instances simultaneously
 - Individual volume control per instance

For longer playing audio, the [Song](#) class is used (but not today...)

Creating Sound Effects



<https://www.bfxr.net>

Pong 11: Audio

- Add sound effects to the game
 - You must have three distinct sounds: "[paddle hit](#)", "[wall hit](#)" and "[score](#)"

You Made it!

Congratulations! You now have your very own working Pong Game made with MonoGame.



Achievement Unlocked!
Completed the tutorial.